

Performance Engineering with

julia

on

Modern Supercomputers



Paderborn Center for
Parallel Computing



NATIONALES
HOCHLEISTUNGS
RECHNEN



PADERBORN
UNIVERSITY



Performance Engineering with Julia on Modern Supercomputers

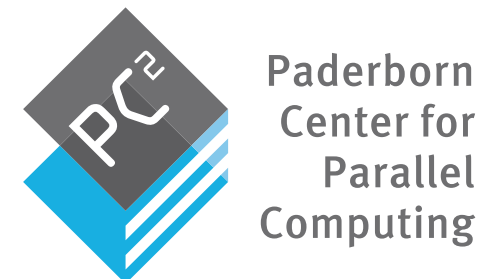
- Performance Engineering Workflow in Julia -

Xin Wu, Alex Wiens, Gerrit Pape, Christian Plessl

Paderborn University, Germany
Paderborn Center for Parallel Computing



Aug 10, 2026, JuliaCon'26, Mainz



- Session 1: Demystifying Modern Supercomputers and Processor Architectures
- Session 2: Performance Engineering Workflow in Julia
- Session 3: Case Study: Molecular Dynamics Simulation of Liquid Argon (ArgonMD.jl)
- inbetween: coffee break

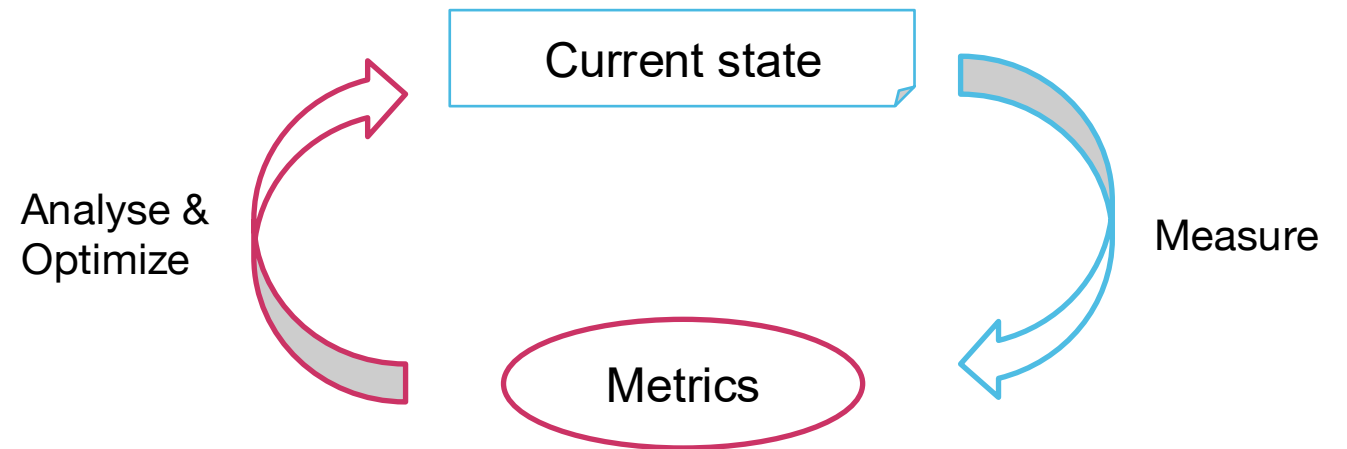
- Introduction to code optimization in Julia
- Profiling
- Benchmarking with BenchmarkTools.jl
- Writing type-stable Julia code
- Hardware performance counters with LIKWID.jl
- Memory optimization and efficient cache utilization
- SIMD vectorization

Summary and Outlook

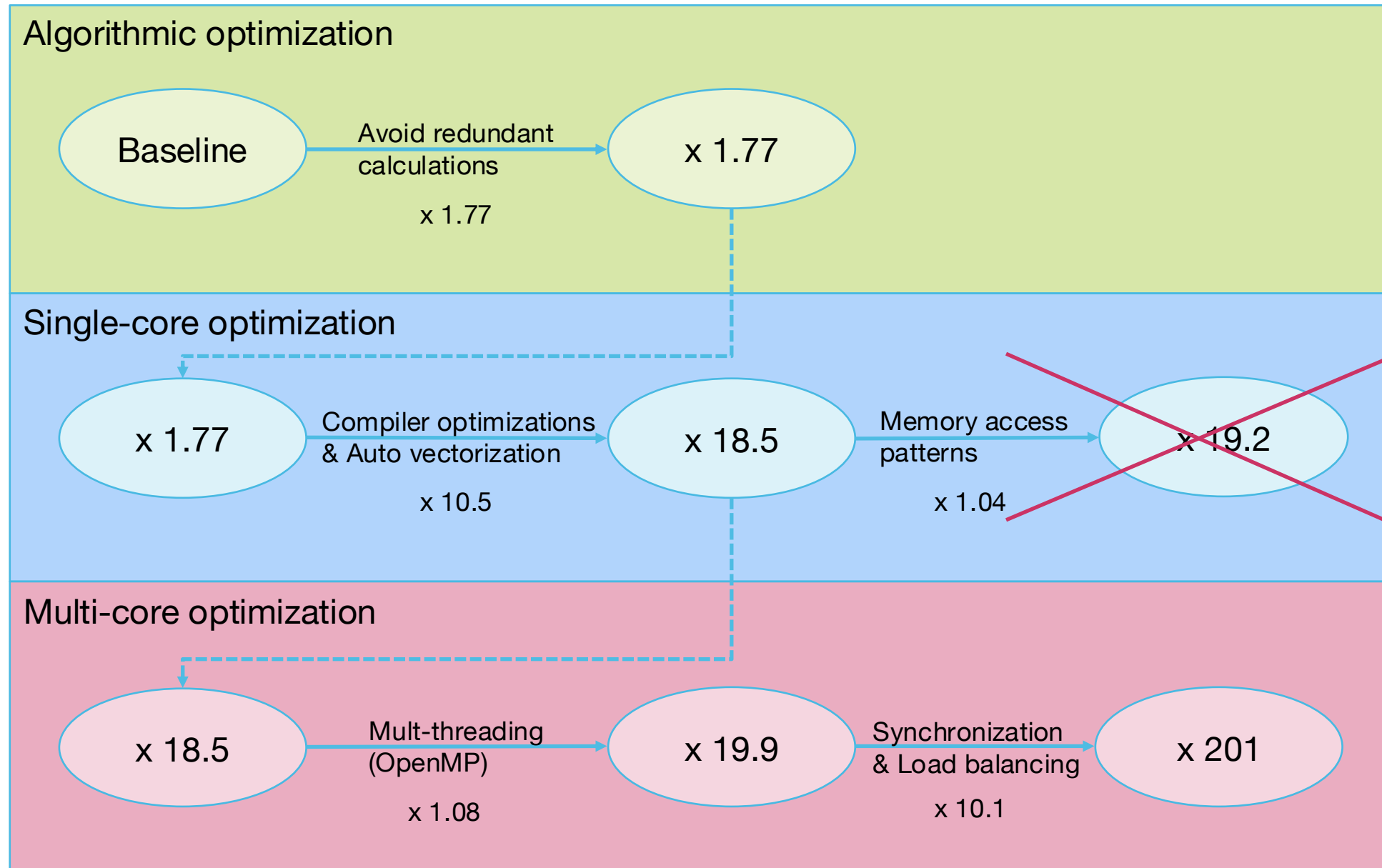
Performance Engineering Process

Performance Engineering Methodology

- Idea: use measurements for systematic analysis and to guide optimization
- Measure runtime (or energy) to create a baseline for comparison
- Identify code regions for optimization by using Profiling
 - e.g. time-intensive functions
- Use further metrics to analyse function behavior
 - improves understanding
 - makes code changes comparable
- Optimize (change) code
- Measure again to compare
- Repeat



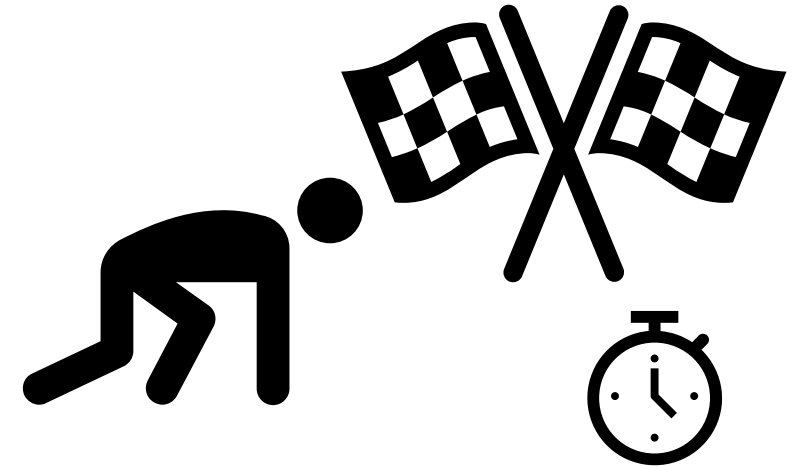
Performance Engineering Process – Iterative process example



Benchmarks

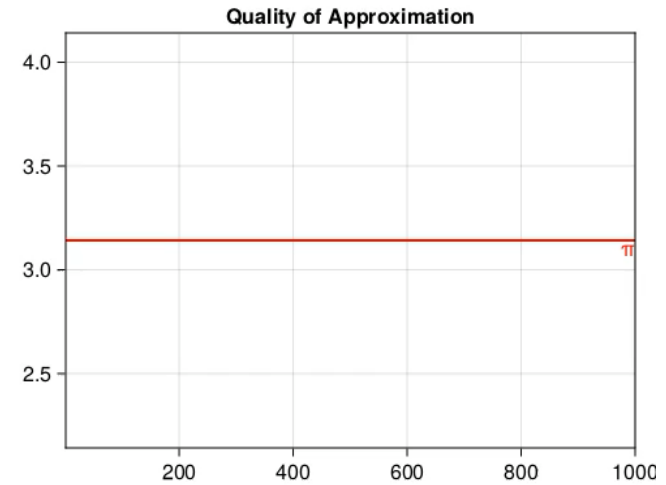
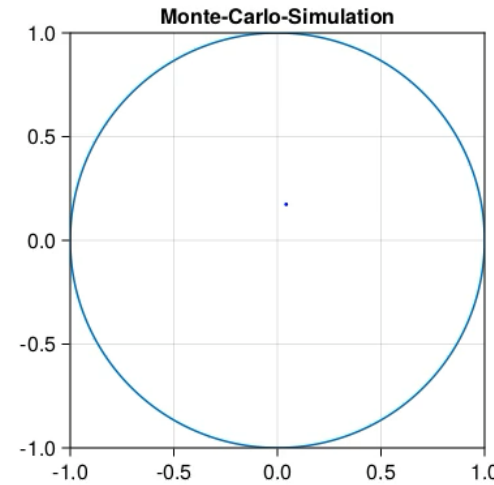
Measuring Performance

- We want to use the hardware of our system efficiently and optimize our programs
 - Therefore, we need to measure the performance of our program to track the impact of our optimizations
 - How can we do that?
-
- **Naïve Approach: Benchmark! Measure the runtime!**
 - Compare different implementations of the same algorithm
 - Compare different systems running the same implementation



Determine π with Monte-Carlo-Simulation

```
function compute_pi(N)
    hits = 0
    for i = 1:N
        x, y = rand(Uniform(-1,1), 2)
        if sqrt(x^2 + y^2) <= 1.0
            hits += 1
        end
    end
    return 4.0 * hits / N
end
```



areas: $A_c = 1^2 \cdot \pi = \pi$ $\rightarrow$ $\frac{A_c}{A_s} = \frac{\pi}{4}$
 $A_s = 2^2 = 4$

$$\tilde{\pi}(n) = 4 \frac{A_c}{A_s} \approx 4 \frac{n_c}{n} \quad \pi = \lim_{n \rightarrow \infty} 4 \frac{n_c}{n}$$

Simple Time Measurement

```
julia> @time compute_pi(2^20)
0.156560 seconds (1.05 M allocations: 80.000 MiB, 75.18% gc time)
```

In the first execution, the function needs first to be **compiled**! So we need to run the code multiple times to get accurate measurements...

```
julia> [@elapsed compute_pi(2^20) for i in 1:5]
5-element Vector{Float64}:
 0.124279209
 0.038146375
 0.041644959
 0.040172583
 0.040120125
```

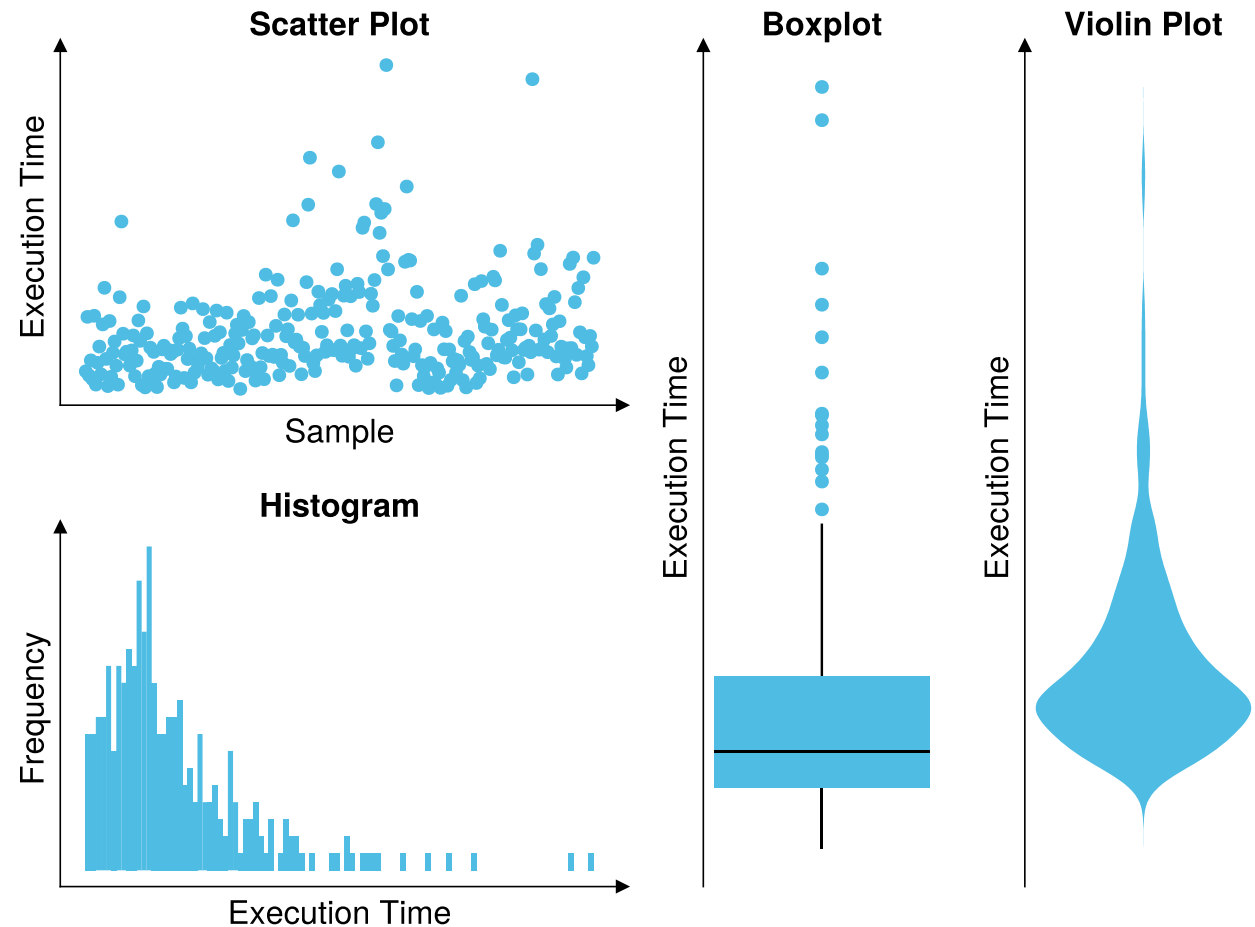
Even after compilation, we can observe **runtime variations** for our execution samples

→ A single runtime measurement is **not sufficient** to get a good understanding of the performance!

Plotting the Data

We executed `compute_pi` several 100 times to get enough(?) data for our performance evaluation

- Plotting the data allows better understanding of sample distribution
- Different types of plots give different insights
- Several plotting packages exist for Julia like Makie, Plots providing simple interfaces for data visualization

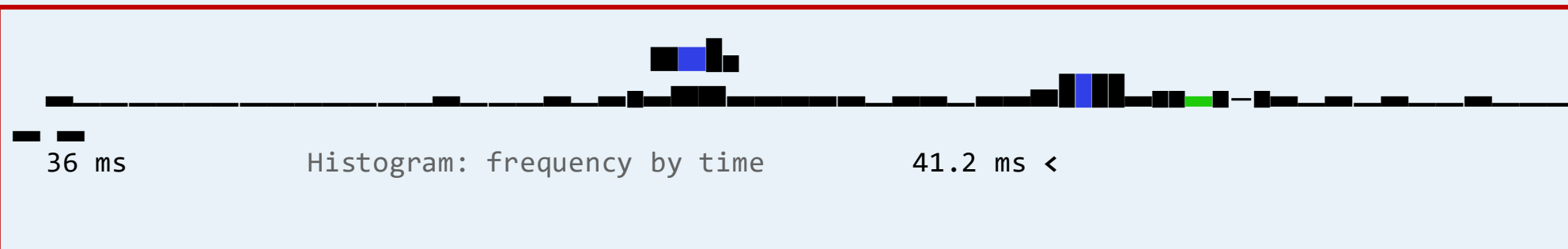


Same mvec! Execution samples plotted in different ways

- Julia package BenchmarkTools simplifies performance evaluation
- @benchmark macro prints comprehensive runtime information
- Measures runtime by finding a tradeoff between total number of samples and total benchmark execution time

Evaluation: Single run of expression
Sample: Time measurements, may consist of multiple evaluations to overcome inaccuracies of used timing method

```
julia> @benchmark compute pi(2^20)
BenchmarkTools.Trial: 126 samples with 1 evaluation.
  Range (min ... max):  36.031 ms ... 121.560 ms
  Time  (median):       39.325 ms
  Time  (mean ± σ):     39.808 ms ±  7.377 ms
                        GC (min ... max):  3.71% ... 69.26%
                        GC (median):        6.30%
                        GC (mean ± σ):     7.21% ±  5.87%
```



Histogram
over all
samples

Memory estimate: 80.00 MiB, allocs estimate: 1048576.

BenchmarkTools Macros

- `@btime` as replacement of `@time` of the Julia standard library prints best execution time and **returns result of expression**
- `@belapsed` as replacement for `@elapsed` **returns best execution time**
- Like `@benchmark`, both macros collect multiple samples and can be configured with the same parameters
- ⚠ When accessing variables in a BenchmarkTools macro, the variable must be accessed with `$variable_name`

```
julia> @btime compute_pi(2^20)
 37.475 ms (1048576 allocations:
80.00 MiB)
3.1396560668945312
```

```
julia> @belapsed compute_pi(2^20)
0.037175292
```

BenchmarkTools Interpolation

- Interpolation using \$ can be used to remove possible measurement artifacts
- Assume we benchmark mvec! and create a new input and output matrix. The buffer allocation will be part of the benchmark measurement:

```
julia> @belapsed fun(zeros(4000),rand(4000,2000),rand(2000))  
0.013009464
```

- Use interpolation to generate buffers before benchmarking:

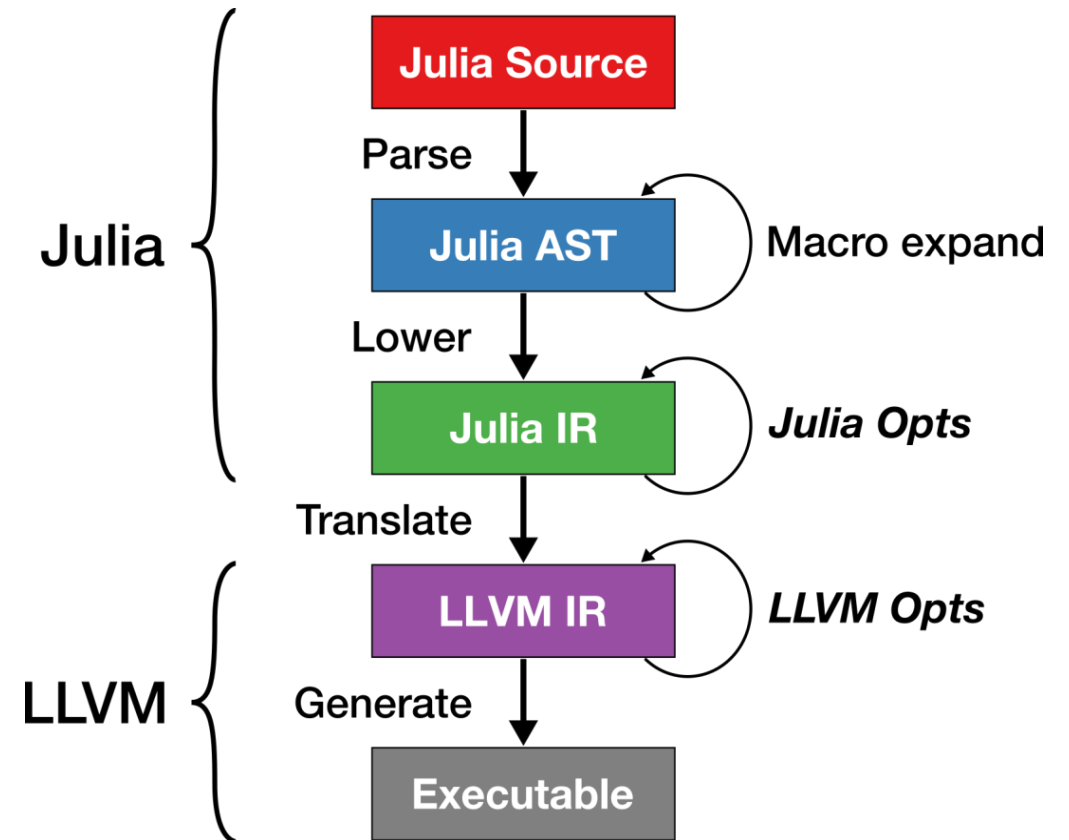
```
julia> @belapsed fun($(zeros(4000)),$(rand(4000,2000)),$(rand(2000)))  
0.00326
```


Type Stability

Julia Compilation Pipeline

Julia compilation pipeline consists of

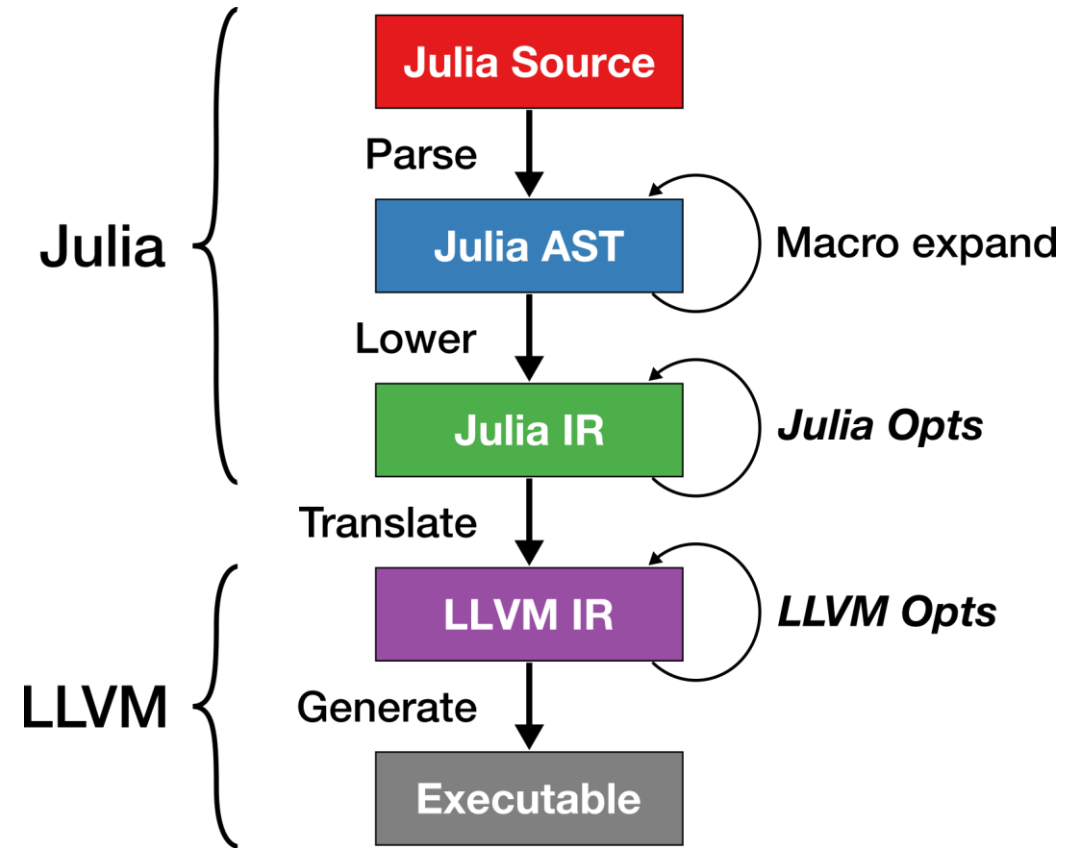
1. parse Julia code into abstract syntax trees (ASTs)
2. Julia ASTs are lowered into Julia IR
3. Julia IR is translated into LLVM IR
4. machine code is generated by LLVM



Julia Compilation Pipeline

Julia compilation pipeline

- parse and macro expand
 - `Meta.parse`
 - `dump`
 - `@macroexpand`
- lowered Julia IR
 - `@code_lowered`
- **type-inferred Julia IR**
 - `@code_typed`
 - `@code_warntype`
- LLVM IR
 - `@code_llvm`
- native code
 - `@code_native`



- **Type stability** is key to performant Julia code.
- A method is type stable, if the data-flow analysis can assign concrete types for all variables in the function and the function return value.

```
# type unstable
> struct TUnstable
    v
end

> bad_example = TUnstable(3);
> bad_example.v
3

> f_mul2(x) = 2 * x.v
f_mul2 (generic function with 1 method)

> @code_typed f_mul2(bad_example)
CodeInfo(
  1 -- %1 = Base.getfield(x, :v)::Any
  |   %2 = (2 * %1)::Any
  |   └─ return %2
) => Any
```

@code_warntype args

- evaluate the arguments and print lowered and type-inferred Julia IR
- **type instability** is warned in **red**
- for checking type stability in method

```
> @code_warntype f_mul2(bad_example)
MethodInstance for f_mul2(::TUnstable)
  from f_mul2(x) @ Main In[31]:1
Arguments
  #self#::Core.Const(f_mul2)
  x::TUnstable
Body::Any
1 - %1 = Base.getproperty(x, :v)::Any
   |   %2 = (2 * %1)::Any
   └──   return %2
```

Type Stability (3/4)

```
# type stable
> struct TStable
    v::Int
end

> good_example = TStable(3)
TStable(3)

> good_example.v
3

> typeof(good_example.v)
Int64

> @code_typed f_mul2(good_example)
CodeInfo(
  1 - %1 = Base.getfield(x, :v)::Int64
    |   %2 = Base.mul_int(2, %1)::Int64
    |   └─ return %2
) => Int64
```

```
> @code_warntype f_mul2(good_example)
MethodInstance for f_mul2(::TStable)
  from f_mul2(x) @ Main In[31]:1
Arguments
  #self#::Core.Const(f_mul2)
  x::TStable
Body::Int64
1 - %1 = Base.getproperty(x, :v)::Int64
  |   %2 = (2 * %1)::Int64
  |   └─ return %2
```

Type Stability (4/4)

```
# type unstable  
> struct TUnstable  
    v  
end
```

- no constraint for v
→ v can be anything
- thus, the size of v is unknown



```
# type stable  
> struct TStable  
    v :: Int  
end
```

- constraint for v
→ v must be `Int`
- thus, the size of v is fixed



Return Type Stability (1/2)

```
function unstable_return_type(v)
  acc = 0
  for i in v
    acc = acc + i
  end
  return acc
end
```

```
> Base.return_types(unstable_return_type, (Vector{Float64},))
```

```
1-element Vector{Any}:
```

```
Union{Float64, Int64}
```

```
> Base.return_types(unstable_return_type, (Vector{Float32},))
```

```
1-element Vector{Any}:
```

```
Union{Float32, Int64}
```



```
function stable_return_type(v)
  acc = zero(eltype(v))
  for i in v
    acc = acc + i
  end
  return acc
end
```

```
> Base.return_types(stable_return_type, (Vector{Float64},))
```

```
1-element Vector{Any}:
```

```
Float64
```

```
> Base.return_types(stable_return_type, (Vector{Float32},))
```

```
1-element Vector{Any}:
```

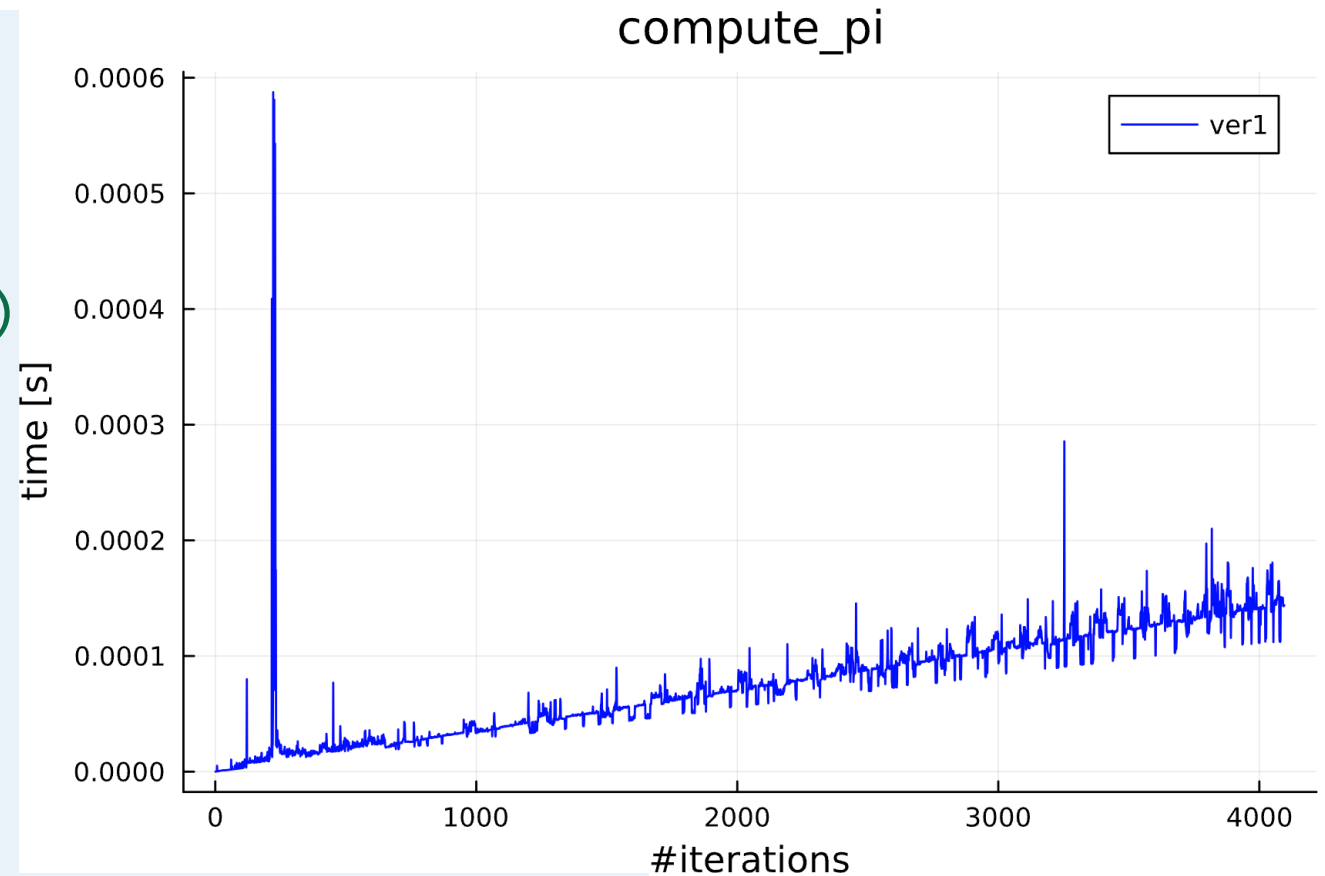
```
Float32
```

Memory Allocations

Example: compute_pi

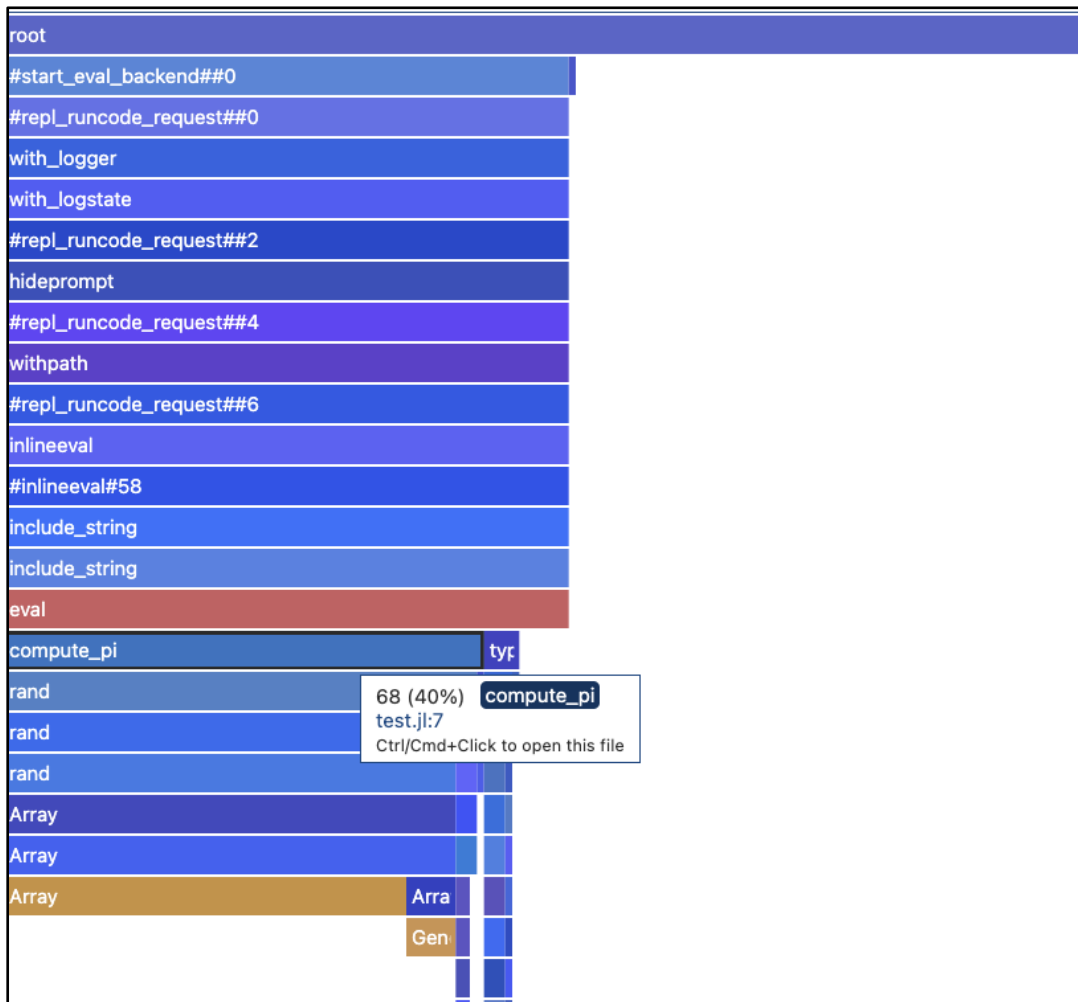
```
function compute_pi(N)
    hits = 0
    for i = 1:N
        x, y = rand(Uniform(-1,1), 2)
        if sqrt(x^2 + y^2) <= 1.0
            hits += 1
        end
    end
    return 4.0 * hits / N
end
```

```
julia> @time compute_pi(500)
0.000026 seconds (500 allocations: 39.062 KiB)
```

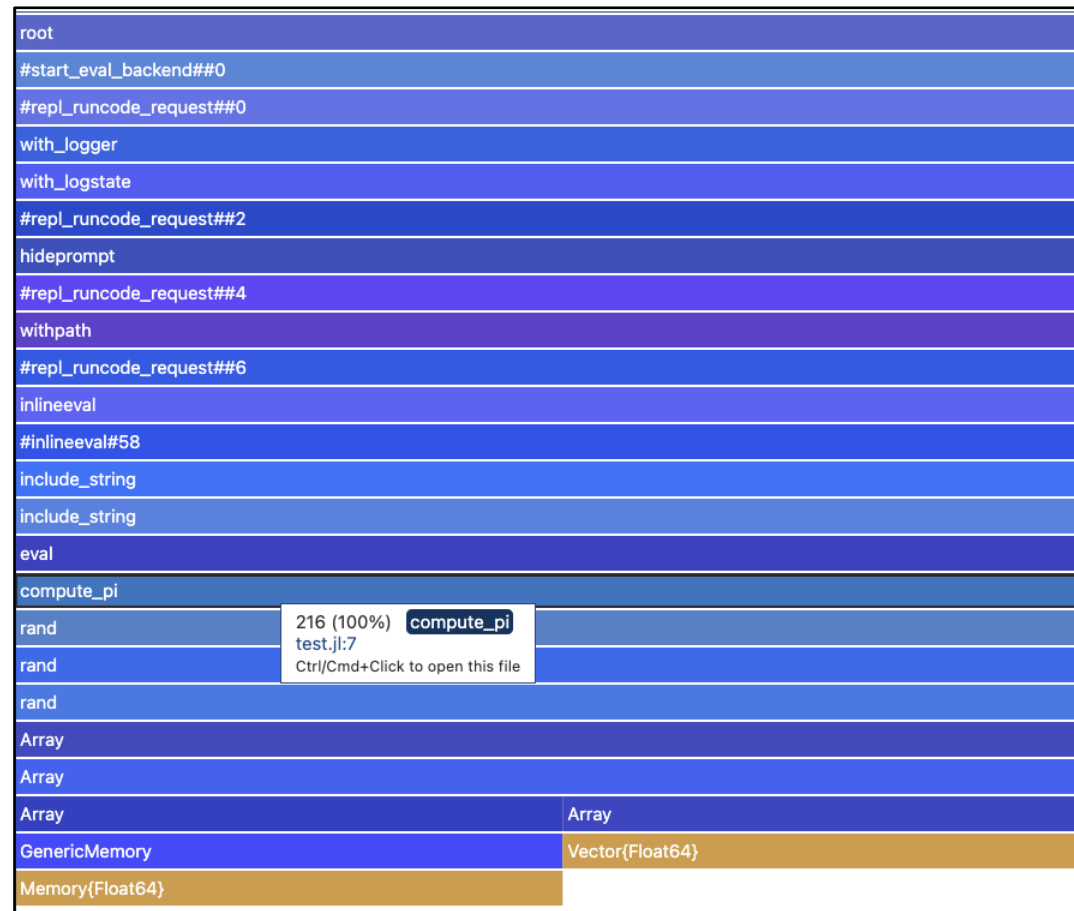


- **Profiling measures code during execution and creates a profile.**
 - Records functions calls and metrics.
 - Used to identify code relevance, performance etc.
- **Different types of profilers**
 - Statistical/Sampling profiler: samples the code interval-based
 - Exact profiler: records everything
- **Metrics**
 - Runtime
 - Allocation count/size
 - other metrics
- **Julia includes multiple profilers**
 - sampling profiler for time and memory
 - allocation tracker
- **Different packages for visualization**

Profiling – FlameGraphs (here in Visual Studio Code)



```
# profile time
@profview compute_pi(2^20)
```



```
# profile allocations
@profview_allocs compute_pi(2^20)
```

Example: compute_pi

```
julia --track-allocation compute_pi.jl

- function compute_pi(N)
-     hits = 0
0     for i = 1:N
40000         x, y = rand(Uniform(-1,1), 2)
0         if sqrt(x^2 + y^2) <= 1.0
0             hits += 1
-         end
0     end
0     return 4.0 * hits / N
- end
-
- compute_pi(500)
```

← Implicitly allocates
Array.

Check the Julia Manual for further profiling techniques
<https://docs.julialang.org/en/v1/manual/profile/>

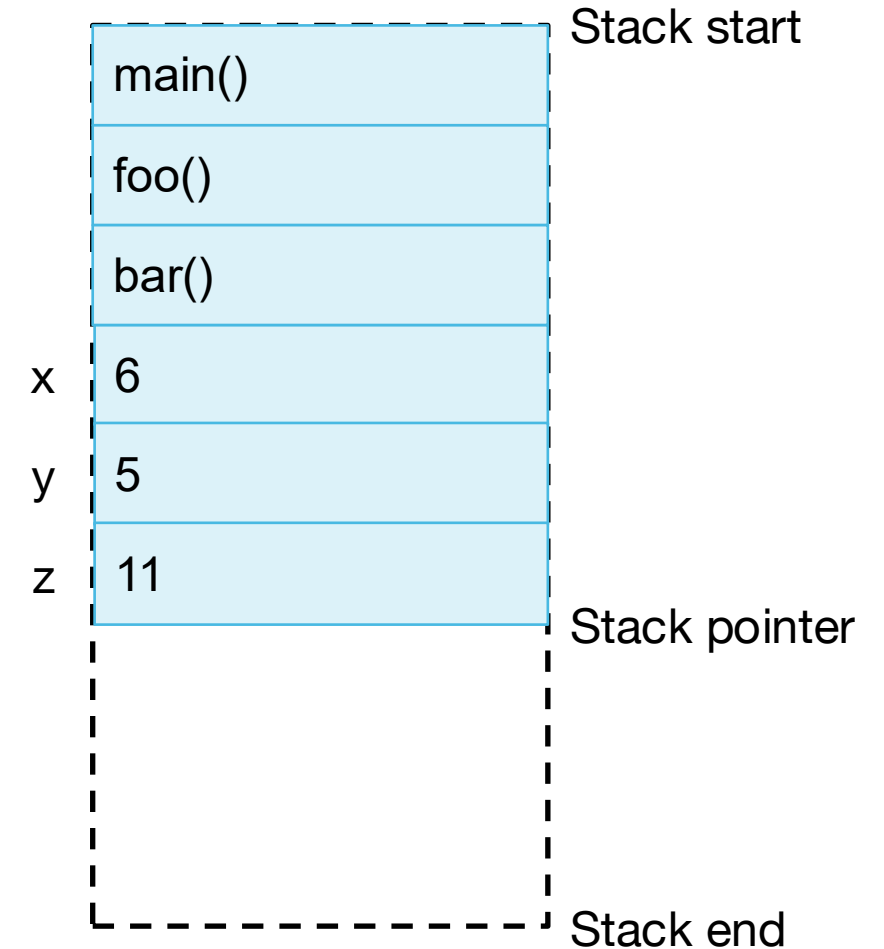
Memory allocation on Stack

- Fixed sized memory area
- Allocation on top of stack
- Deallocation in reverse order
- Use:
 - temporary data
 - function frames (parameters, return address)

```
function bar(x)
  y = 5
  z = x + y
  return z
end

function foo()
  bar(6)
end

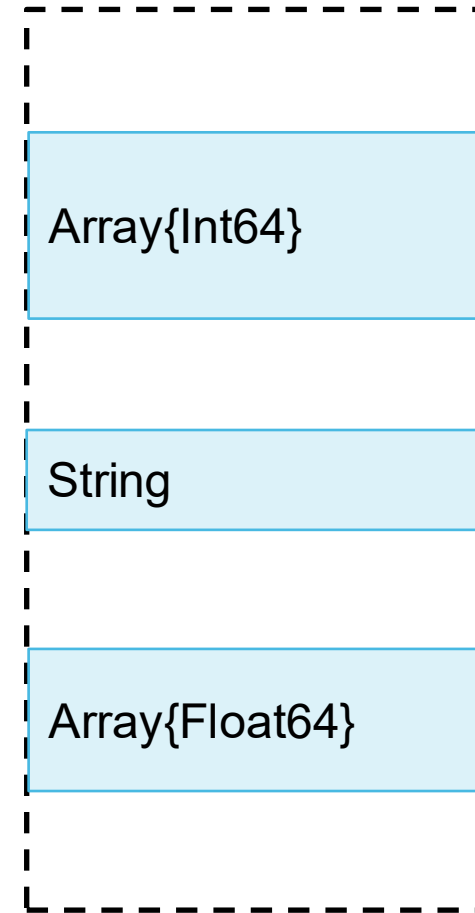
foo()
```



Stack allocation

Memory allocation on Heap

- Memory area managed by Julia
- Order of (de-) allocation does not matter
- Might need to ask OS for more memory
- Garbage Collector regularly cleans up
- Use:
 - large data
 - data persistence between function calls
 - unpredictable data structures
- Stack allocation:
 - small overhead
 - limited by maximal stack size
- Heap allocation:
 - substantial overhead
 - limited by free system memory



Heap allocation

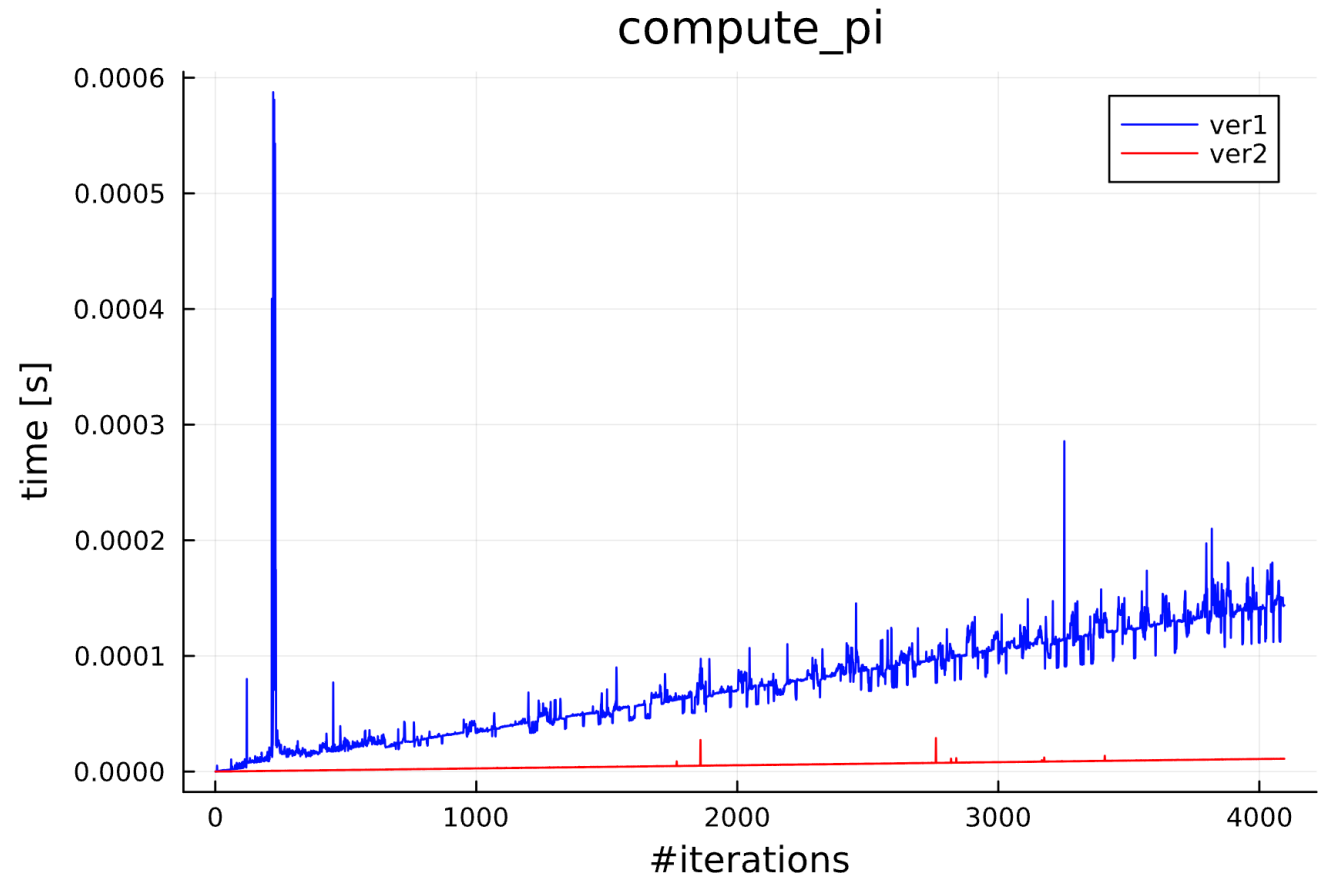
Allocations

Implicitly allocates Array on Heap

```
x, y = rand(Uniform(-1,1), 2)
```

Saves values directly on the stack

```
x, y = rand(Uniform(-1,1)),  
        rand(Uniform(-1,1))
```



Allocation decision depends on concrete Julia version

Requirements for stack usage:

- Data fits on stack
- Data is referenced only while allocation persists (function call is going on)
- Data has fixed size known at compile time
- Data is immutable

Avoid heap allocation

@noinline for easier reasoning.

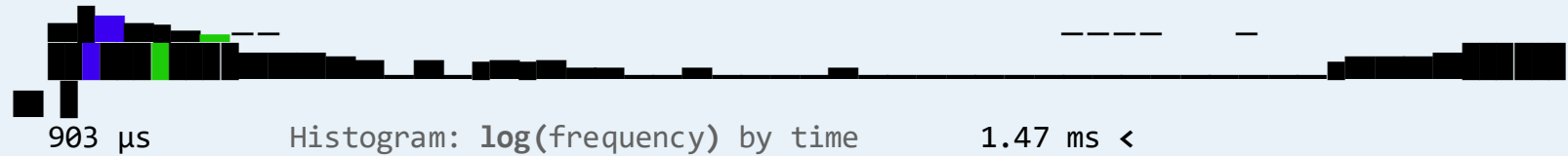


```
@noinline function grey(x)
    mx = mean(x)
    return [mx,mx,mx]
end

function fadetogrey!(img)
    for i = 1:3:length(img)
        img[i:i+2] .= grey(img[i:i+2])
    end
end

img = rand(Float32, 60000)
@benchmark fadetogrey!(img)
```

Range (min ... max):	902.584 μs ... 1.622 ms	GC (min ... max):	0.00% ... 33.36%
Time (median):	921.333 μs	GC (median):	0.00%
Time (mean ± σ):	962.133 μs ± 133.326 μs	GC (mean ± σ):	3.70% ± 9.03%



Memory estimate: 3.05 MiB, allocs estimate: 40000.



A lot of allocations for
turning an image to grey.

Avoid heap allocation

@noinline for easier reasoning.



```
@noinline function grey(x)
    mx = mean(x)
    return [mx,mx,mx]
end

function fadetogrey!(img)
    for i = 1:3:length(img)
        img[i:i+2] .= grey(img[i:i+2])
    end
end

img = rand(Float32, 60000)
@time fadetogrey!(img)
@time fadetogrey!(img)
```

```
julia --track-allocation grey.jl
0.131603 seconds (624.64 k allocations: 33.150 MiB, 4.24% gc time, 80.23%
compilation time)
0.026449 seconds (40.00 k allocations: 3.052 MiB)
```

```
- @noinline function grey(x)
0   mx = mean(x)
3200000   return [mx,mx,mx]
-   end
-
- function fadetogrey!(img)
0   for i = 1:3:length(img)
3200000       img[i:i+2] .= grey(img[i:i+2])
0   end
-   end
-
- img = rand(Float32, 60000);
- @time fadetogrey!(img);
- @time fadetogrey!(img);
-
-
-
```

← Array allocation
for return pixel.

← Array allocation
for parameter
pixel.

Avoid heap allocation

@noinline for easier reasoning.



```
@noinline function grey(x)
    mx = mean(x)
    return (mx,mx,mx)
end

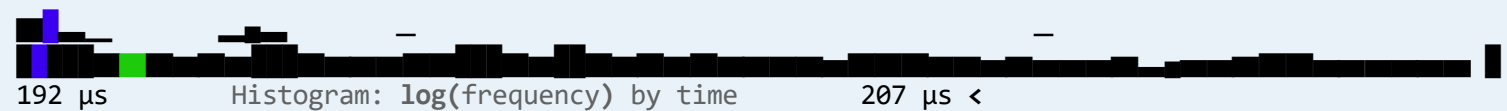
function fadetogrey!(img)
    for i = 1:3:length(img)
        img[i:i+2] .= grey(view(img, i:i+2))
    end
end

img = rand(Float32, 60000)
@benchmark fadetogrey!(img)
```

← Return tuple will be allocated on stack.

← Pass view into Array to function.

Range (min ... max):	192.041 μs ... 411.625 μs	GC (min ... max):	0.00% ... 0.00%
Time (median):	192.333 μs	GC (median):	0.00%
Time (mean ± σ):	193.556 μs ± 4.195 μs	GC (mean ± σ):	0.00% ± 0.00%



Memory estimate: 0 bytes, allocs estimate: 0.

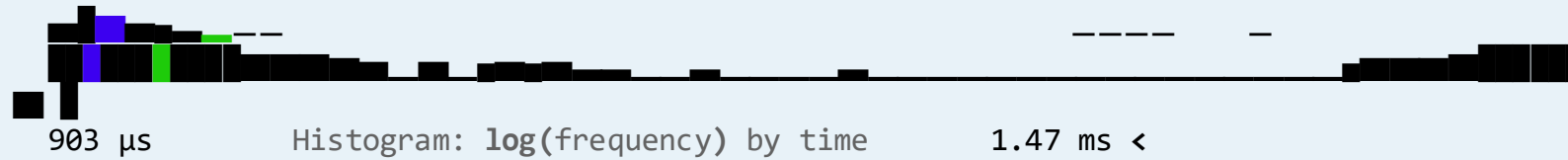


No allocations anymore.

Avoid heap allocation

Before:

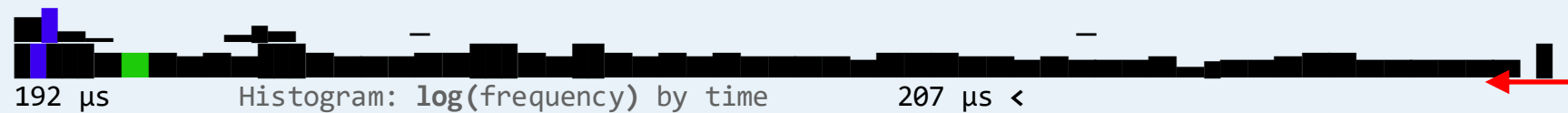
```
Range (min ... max): 902.584 µs ... 1.622 ms | GC (min ... max): 0.00% ... 33.36%
Time (median): 921.333 µs | GC (median): 0.00%
Time (mean ± σ): 962.133 µs ± 133.326 µs | GC (mean ± σ): 3.70% ± 9.03%
```



Memory estimate: 3.05 MiB, allocs estimate: 40000.

After:

```
Range (min ... max): 192.041 µs ... 411.625 µs | GC (min ... max): 0.00% ... 0.00%
Time (median): 192.333 µs | GC (median): 0.00%
Time (mean ± σ): 193.556 µs ± 4.195 µs | GC (mean ± σ): 0.00% ± 0.00%
```



Memory estimate: 0 bytes, allocs estimate: 0.

← No Garbage collection activity.

← Faster and less variance.

← No allocations

Avoid heap allocation – Another improvement

@noinline for easier reasoning.



```
@noinline function grey!(x)
    mx = mean(x)
    x[1] = mx
    x[2] = mx
    x[3] = mx
end
```

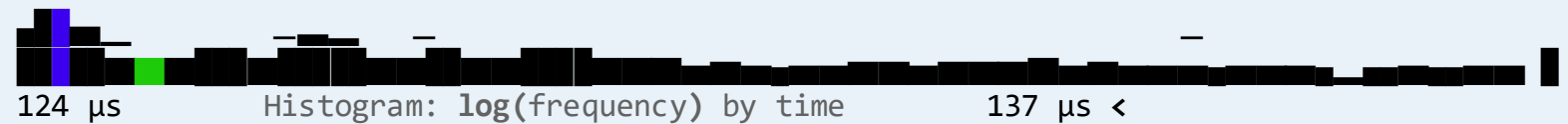
← Directly mutate values.

```
function fadetogrey!(img)
    for i = 1:3:length(img)
        grey!(view(img, i:i+2))
    end
end
```

← Pass view into Array to function.

```
img = rand(Float32, 60000)
b = @benchmark fadetogrey!($img)
```

Range (min ... max):	123.916 μs ... 224.083 μs	GC (min ... max):	0.00% ... 0.00%
Time (median):	124.417 μs	GC (median):	0.00%
Time (mean ± σ):	125.260 μs ± 3.067 μs	GC (mean ± σ):	0.00% ± 0.00%



Memory estimate: 0 bytes, allocs estimate: 0.

Avoid heap allocation: Static Arrays/ Broadcasting

- Static Arrays

- New type that “looks” like Array, but is a Tuple
- Size is always known at compile time and Tuple is allocated on Stack

```
function grey(x)
    mx = mean(x)
    return [mx, mx, mx]
end
```

use SVector{3,T}



```
using StaticArrays
function grey(x)
    mx = mean(x)
    return SA[mx, mx, mx]
end
```

- Broadcasting/ Dot operator

- Apply function to every element
- Avoid allocation of temporary arrays
- Use broadcast! or dot syntax
 - dot prefix or @. macro

```
a,b,c = rand(2^20),rand(2^20),rand(2^20)
```

```
@benchmark 6a + 3b + 5c
```

```
Range (min ... max): 2.896 ms ... 25.982 ms | GC (min ... max): 0.00% ... 77.59%
Memory estimate: 32.00 MiB, allocs estimate: 8.
```

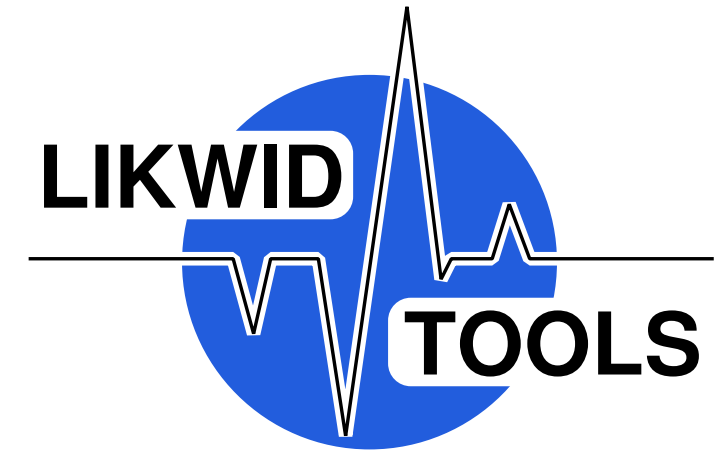
```
@benchmark 6 .* a .+ 3 .* b .+ 5 .* c
```

```
Range (min ... max): 867.625 μs ... 6.137 ms | GC (min ... max): 0.00% ... 79.60%
Memory estimate: 8.00 MiB, allocs estimate: 7.
```


Performance Measurement

LIKWID and Performance Counters

- Modern CPUs provide special registers to count low-level instructions like floating-point operations (FLOPs) or memory accesses
- Allows collection of performance data with low overhead during execution
- LIKWID (Like I Knew What I'm Doing) is a tool to utilize these registers without detailed knowledge
- We can use the Julia package `LIKWID.jl` to make use of these counters



<https://hpc.fau.de/research/tools/likwid/>



<https://github.com/JuliaPerf/LIKWID.jl>

Example: Matrix element-wise Addition

- Our matrix addition contains one FLOPs per element
- So the total number of FLOPs can be calculated with $FLOPs = n \cdot m$ for $A \in \mathbb{R}^{n \times m}$
- For more complex applications, determining the FLOPs may be a bit more complicated
- Performance counters can be used to count these operations during execution

```
function mat_add!(c::Matrix, a::Matrix, b::Matrix)
    for row in 1:size(a,1)
        for col in 1:size(a,2)
            c[row,col] = a[row,col] + b[row,col]
        end
    end
    return c
end
```



One floating-point operations per element in matrix A

Example: FLOPS of Matrix element-wise Addition

The macro `@perfmon` sets up the counters for a group and returns tables with the **raw counter data** and **derived metrics**

```
using LIKWID
```

```
A = rand(1000, 1000) # 10^6 elements
```

```
B = rand(1000, 1000)
```

```
C = zeros(1000, 1000)
```

```
@perfmon "FLOPS_DP" mat_add!(C, A, B)
```

Group: FLOPS_DP

Event	Thread 1
ACTUAL_CPU_CLOCK	1.06987e7
MAX_CPU_CLOCK	7.43822e6
RETIRED_INSTRUCTIONS	1.00827e7
CPU_CLOCKS_UNHALTED	1.0591e7
RETIRED_SSE_AVX_FLOPS_ALL	1.0e6
MERGE	0.0

Raw
Counters

Metric	Thread 1
Runtime (RDTSC) [s]	0.00301974
Runtime unhaltd [s]	0.00436703
Clock [MHz]	3523.75
CPI	1.05042
DP [MFLOP/s]	331.154

Derived
Metrics

Model FLOPS: $1000 * 1000 = 1,00,000$

→ Our simple model perfectly matches the measurement!

- LIKWID provides different performance groups
- Every group uses a different set of counters to calculate specific derived metrics
- Available groups depend on exact CPU architecture and model

```
julia> LIKWID.PerfMon.supported_groups()
```

```
Dict{String, LIKWID.GroupInfoCompact} with 19 entries:
```

"L2CACHE"	=> L2CACHE => L2 cache miss rate/ratio (experimental)
"MEM"	=> MEM => Main memory read bandwidth in MBytes/s
"BRANCH"	=> BRANCH => Branch prediction miss rate/ratio
"FLOPS_SP"	=> FLOPS_SP => Single-precision MFLOP/s
"FLOPS_ANY"	=> FLOPS_ANY => MFLOP/s
"DIVIDE"	=> DIVIDE => Divide unit information
"CPI"	=> CPI => Cycles per instruction
"L3CACHE"	=> L3CACHE => L3 cache miss rate/ratio (experimental)
"CACHE"	=> CACHE => Data cache miss rate/ratio
"ICACHE"	=> ICACHE => Instruction cache miss rate/ratio
"TLB"	=> TLB => TLB miss rate/ratio
"CLOCK"	=> CLOCK => Cycles per instruction
"TMA"	=> TMA => Pipeline Utilization Analysis
"FLOPS_DP"	=> FLOPS_DP => Double-precision MFLOP/s
"ENERGY"	=> ENERGY => Power and Energy consumption
"FLOPS_BF16"	=> FLOPS_BF16 => Bfloat16 MFLOP/s
"DATA"	=> DATA => Load to store ratio
"MEMREAD"	=> MEMREAD => Main memory read bandwidth in MBytes/s
"MEMWRITE"	=> MEMWRITE => Main memory write bandwidth in MBytes/s

Memory Layout

Determine Effect of Memory Access Pattern

- Compute sum of two matrices

```
c[row,col] = a[row,col] + b[row,col]
```

- Two implementations with with double nested loops
 - mat_add_col! iterate column-by-column
 - mat_add_row! iterate row-by-row
 - operation memory bound (OI = 1 FLOP / 24 bytes)
 - identical matrix sizes
- Expected execution time?
 - should be identical for both versions since we have the same number of operations

```
function mat_add_col!(c::Matrix, a::Matrix, b::Matrix)
    for row in 1:size(a,1)
        for col in 1:size(a,2)
            c[row,col] = a[row,col] + b[row,col]
        end
    end
    return c
end

function mat_add_row!(c::Matrix, a::Matrix, b::Matrix)
    for col in 1:size(a,2)
        for row in 1:size(a,1)
            c[row,col] = a[row,col] + b[row,col]
        end
    end
    return c
end
```

Benchmark implementations

Only three executions

mat_add_col!

~ 1.8s

```
julia> @benchmark mat_add_col!(C, A, B)
BenchmarkTools.Trial: 3 samples with 1 evaluation per sample.
Range (min ... max): 1.878 s ... 1.881 s | GC (min ... max): 0.00% ... 0.00%
Time (median): 1.880 s | GC (median): 0.00%
Time (mean ± σ): 1.880 s ± 1.246 ms | GC (mean ± σ): 0.00% ± 0.00%
```

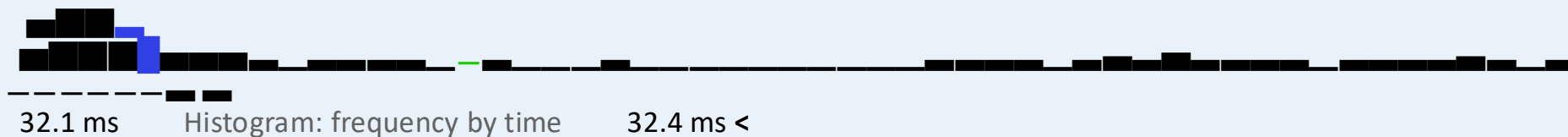


Memory estimate: 0 bytes, allocs estimate: 0.

mat_add_row!

~ 0.032s

```
julia> @benchmark mat_add_row!(C, A, B)
BenchmarkTools.Trial: 156 samples with 1 evaluation per sample.
Range (min ... max): 32.066 ms ... 33.360 ms | GC (min ... max): 0.00% ... 0.00%
Time (median): 32.087 ms | GC (median): 0.00%
Time (mean ± σ): 32.147 ms ± 140.197 μs | GC (mean ± σ): 0.00% ± 0.00%
```



Memory estimate: 0 bytes, allocs estimate: 0.

```
s = 2^13
A = rand(s,s)
B = rand(s,s)
C = zeros(s,s)
```


Measure L2 Cache activity

mat_add_col!

L2 accesses:

637'455'000

mat_add_row!

L2 accesses:

12'967'100

LIKWID.@perfmon "L2CACHE" mat_add_col!(C, A, B)

Metric	Thread 1
Runtime (RDTSC) [s]	3.05303
Runtime unhaltd [s]	6.17223e-10
L2 request rate	7.74706
L2 miss rate	4.54202
L2 miss ratio	0.586289
L2 accesses	6.37455e8
L2 hits	2.63722e8
L2 misses	3.73733e8

LIKWID.@perfmon "L2CACHE" mat_add_row!(C, A, B)

Metric	Thread 1
Runtime (RDTSC) [s]	0.0521567
Runtime unhaltd [s]	6.17223e-10
L2 request rate	0.946445
L2 miss rate	0.701709
L2 miss ratio	0.741416
L2 accesses	1.29671e7
L2 hits	3.3531e6
L2 misses	9.61403e6

Memory Layout of Matrices

a11	a12	a13	a14
a21	a22	a23	a24
a31	a32	a33	a34
a41	a42	a43	a44

matrix data structure

	a11	a12	a13	a14	a21	a22	a23	a24	...
--	-----	-----	-----	-----	-----	-----	-----	-----	-----

row-major order

	a11	a21	a31	a41	a12	a22	a32	a42	...
--	-----	-----	-----	-----	-----	-----	-----	-----	-----

column-major order

how data is placed in (linear) main memory

- **Convention how to place multi-dimensional arrays in main memory**
 - row-major order: used by C, Pascal (system programming languages)
 - column-major order: used by Julia, Fortran, Matlab (technical programming languages)

View from Perspective of Cache

index	meta	cache line, with 8 blocks w/ 8 bytes each							
100		a11	a21	a31	a41	a12	a22	a23	a24
101		a31	a32	a33	a34	a41	a42	a43	a44
102									

L1 cache after all elements of a have been read once (schematic)

- Initially, no element of Matrix a is in cache
 - first access to matrix element e.g. $a[1,1]$ will result in cache miss
 - processor stalls (miss penalty) until cache controller has **fetches a complete block** from the L2 cache
 - when code accesses $a[2,1]$, the corresponding value is already in cache (1 miss, 8 hits)
- Linear access allow memory prefetcher to predict next access with high accuracy

View from Perspective of Cache (2)

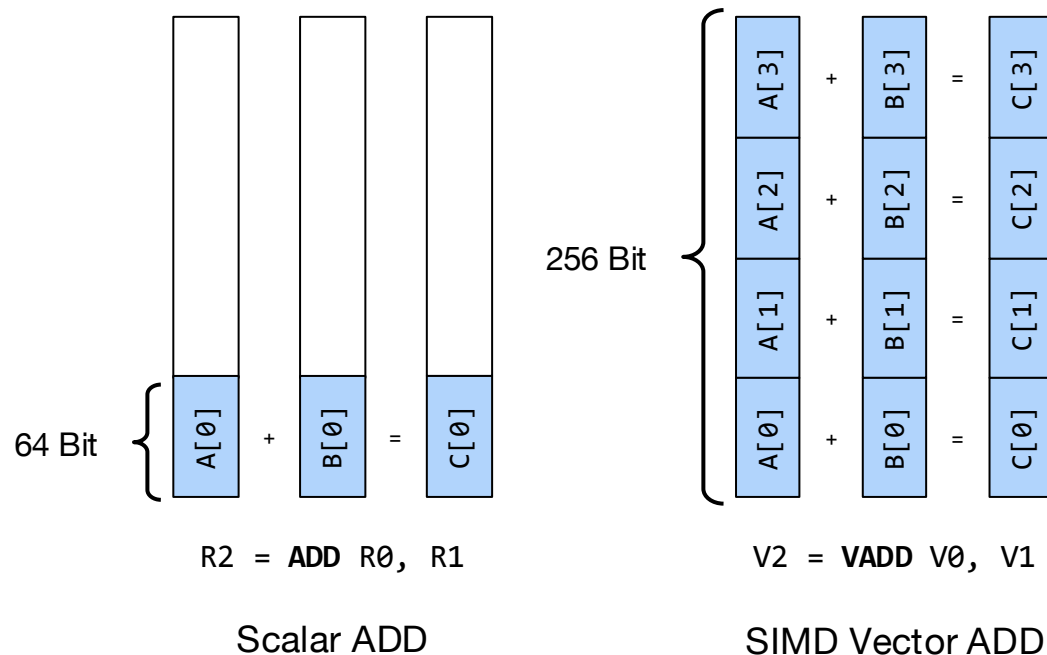
- If elements are accessed row-wise
 - also miss on first access, then 2 elements can be found in cache line (hits)
 - next access will cause miss again
- No problem for small matrices that fit completely into cache
 - after a few misses, the complete matrix is in cache
 - but for large matrices that do not fit in cache, there is a significant performance hit because of repeated misses
- Derived optimization strategy
 - refactor program to access data in preferred direction (column-wise) whenever possible
 - consider storing matrices in transpose form

index	meta	cache line, with 8 blocks w/ 8 bytes each							
100		a11	a21	a31	a41	a12	a22	a32	a42
101		a31	a32	a33	a34	a41	a42	a43	a44
102									

SIMD

Single Instruction Multiple Data (SIMD)

- Concurrent execution of the same operation on wider (vector) registers
 - data-level parallelism
- Extension to normal (scalar) CPU instructions
 - Scalar registers (64 bit = 1 double)
 - AVX2 registers (256 bit = 4 double)
 - AVX512 registers (512 bit = 8 double)
 - Available on Otus



Example: adding two registers in double precision

- `@inbounds` removes bound checks
 - but might lead to illegal memory access
- `@simd` allows Julia to optimize simple loops
 - assumes that iteration order can be changed
- `@turbo` might help in certain cases
- can produce numerically differing results
 - different order of floating point operations
 - relaxed accuracy requirements („fast math“)
- use with care



Example: dot product

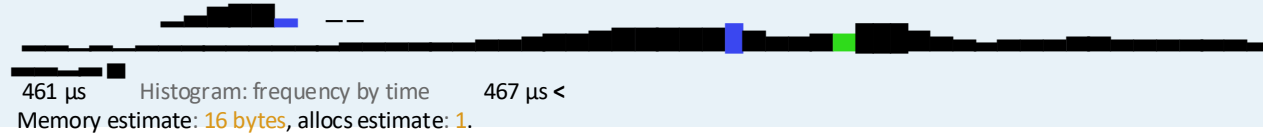
```
function vector_dot(B, C)
    a = zero(eltype(B))
    for i in eachindex(B,C)
        @inbounds a += B[i] * C[i]
    end
    return a
end

function vector_dot_macro(B, C)
    a = zero(eltype(B))
    @simd for i in eachindex(B,C)
        @inbounds a += B[i] * C[i]
    end
    return a
end

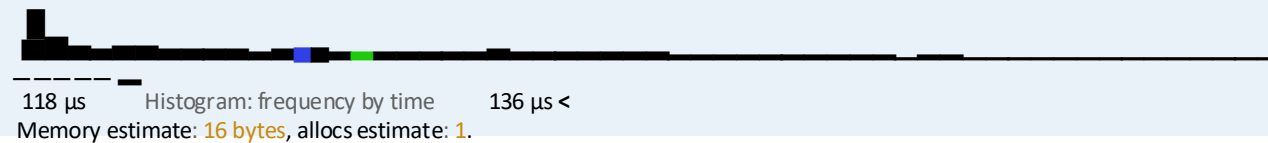
using LoopVectorization
function vector_dot_turbo(B, C)
    a = zero(eltype(B))
    @turbo for i in eachindex(B,C)
        @inbounds a += B[i] * C[i]
    end
    return a
end
```

Example: dot product – Benchmarks (Otus)

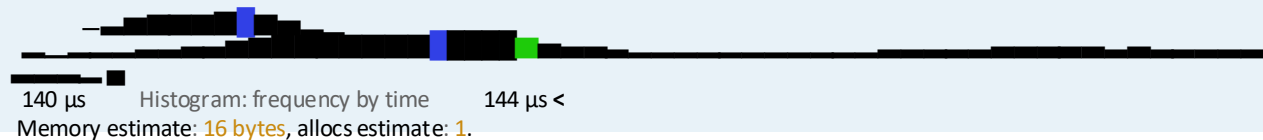
```
julia> @benchmark vector_dot(B,C)
BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.
Range (min ... max): 460.914 μs ... 1.629 ms | GC (min ... max): 0.00% ... 0.00%
Time (median): 464.110 μs | GC (median): 0.00%
Time (mean ± σ): 464.640 μs ± 13.709 μs | GC (mean ± σ): 0.00% ± 0.00%
```



```
julia> @benchmark vector_dot_macro(B,C)
BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.
Range (min ... max): 118.419 μs ... 732.855 μs | GC (min ... max): 0.00% ... 0.00%
Time (median): 121.884 μs | GC (median): 0.00%
Time (mean ± σ): 122.867 μs ± 7.263 μs | GC (mean ± σ): 0.00% ± 0.00%
```



```
julia> @benchmark vector_dot_turbo(B,C)
BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.
Range (min ... max): 139.690 μs ... 1.238 ms | GC (min ... max): 0.00% ... 0.00%
Time (median): 140.972 μs | GC (median): 0.00%
Time (mean ± σ): 141.348 μs ± 11.003 μs | GC (mean ± σ): 0.00% ± 0.00%
```



```
julia> using LinearAlgebra; @benchmark dot(B,C)
BenchmarkTools.Trial: 10000 samples with 1 evaluation per sample.
Range (min ... max): 28.263 μs ... 2.724 ms | GC (min ... max): 0.00% ... 0.00%
Time (median): 30.957 μs | GC (median): 0.00%
Time (mean ± σ): 31.517 μs ± 27.562 μs | GC (mean ± σ): 0.00% ± 0.00%
```



```
function vector_dot(B, C)
    a = zero(eltype(B))
    for i in eachindex(B,C)
        @inbounds a += B[i] * C[i]
    end
    return a
end
```

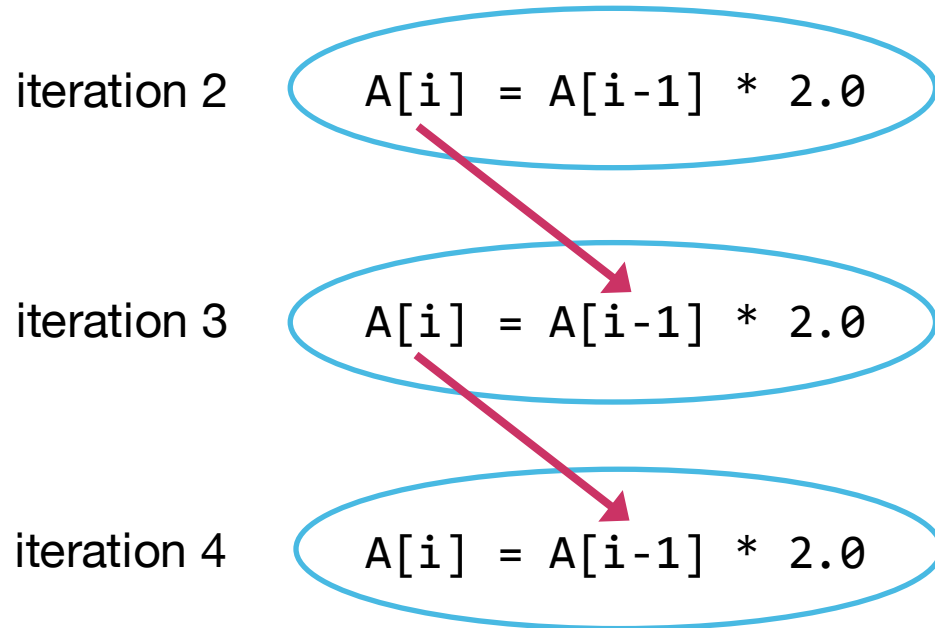
```
function vector_dot_macro(B, C)
    a = zero(eltype(B))
    @simd for i in eachindex(B,C)
        @inbounds a += B[i] * C[i]
    end
    return a
end
```

```
using LoopVectorization
function vector_dot_turbo(B, C)
    a = zero(eltype(B))
    @turbo for i in eachindex(B,C)
        @inbounds a += B[i] * C[i]
    end
    return a
end
```

uses optimized
BLAS library

Example: loop with loop-carried dependency

- each iteration depends on result of previous iteration



```
function test!(A)
    for i in 2:length(A)
        @inbounds A[i] = A[i-1] * 2.0
    end
end

function test_simd!(A)
    @simd for i in 2:length(A)
        @inbounds A[i] = A[i-1] * 2.0
    end
end

function test_turbo!(A)
    @turbo for i in 2:length(A)
        @inbounds A[i] = A[i-1] * 2.0
    end
end
```

Example: loop with loop-carried dependency

```
> A = rand(5)
> B = copy(A)
> C = copy(A)
> D = copy(A)

> test!(B)
> test_simd!(C)
> test_turbo!(D)

> @show isapprox(B, C)
isapprox(B, C) = true

> @show isapprox(B, D)
isapprox(B, D) = false
```

```
function test!(A)
    for i in 2:length(A)
        @inbounds A[i] = A[i-1] * 2.0
    end
end

function test_simd!(A)
    @simd for i in 2:length(A)
        @inbounds A[i] = A[i-1] * 2.0
    end
end

function test_turbo!(A)
    @turbo for i in 2:length(A)
        @inbounds A[i] = A[i-1] * 2.0
    end
end
```

@simd checks for loop-carried dependencies while @turbo might violate correctness.

Summary

- Performance engineering is an iterative process
 - measure performance
 - identify bottleneck
 - optimize
 - measure performance and compare
- Julia allows for high performance, but one needs to avoid common pitfalls.
- Code needs to be adjusted to hardware to utilize full performance.
- Most performance in modern systems is offered by parallel calculations.

- **Node**
 - Multiple processes on different compute nodes (MPI)
- **Process**
 - Multiple processes in parallel (shared memory, MPI, SPMD)
- **Thread**
 - Multiple threads in parallel on multiple cores (Pthread, OpenMP)
- **Instruction**
 - Multiple instructions in parallel (CPU: Superscalarity, Pipelining)
- **Data**
 - Same instruction on multiple data elements (SIMD)

Serial code optimization

Measured problem	Solution
@warncode shows Type Instability	Type annotation Improve (Return) Type stability
Heap allocations in profiling	StaticArrays Remove unnecessary array allocations
Waiting on File I/O	Reduce redundant data
Cache Misses	Memory Layout Optimization, AoS to SoA, Cache blocking
Ratio of scalar to packed instructions	SIMD friendly code transformation
BranchPrediction Misses	Code transformation
Top-down Microarchitecture Analysis	Pipeline Utilization Optimization

Multithreading/Distributed optimization

Measured problem	Solution
Multithreading: High memory transfer between NUMA domains	Improve data locality wrt. NUMA domains
High scheduling/task creation overhead	Increase task workload, Reduce number of threads
Unbalanced core utilization	Workload balancing
Unbalanced communication	Improve workload distribution Communication pattern
Waiting in locks	Improve thread synchronization (e.g. Lock freedom)
Waiting for communication	Transform communication pattern Latency hiding/Computation-Communication-Overlap

